

'QUIT' string. A function that returns values of a single data type is easier to understand than a function that can return values of many possible types. I discussed this in “Return Values Should Always Have the Same Data Type” on page 177.

Between the three towers, only six to-from tower combinations are possible. Despite the fact that we hardcoded all six values in the condition that checks the move, the code is much easier to read than something like `len(response) != 2 or response[0] not in 'ABC' or response[1] not in 'ABC' or response[0] == response[1]`. Given these circumstances, the hardcoding approach is the most straightforward.

Generally, it's considered bad practice to hardcode values such as "AB", "AC", and other values as magic values, which are valid only as long as the program has three poles. But although we might want to adjust the number of disks by changing the `TOTAL_DISKS` constant, it's highly unlikely that we'll add more poles to the game. Writing out every possible pole move on this line is fine.

We create two new variables, `fromTower` and `toTower`, as descriptive names for the data. They don't serve a functional purpose, but they make the code easier to read than `response[0]` and `response[1]`:

```
# Use more descriptive variable names:
fromTower, toTower = response[0], response[1]
```

Next, we check whether or not the selected towers constitute a legal move:

```
if len(towers[fromTower]) == 0:
    # The "from" tower cannot be an empty tower:
    print("You selected a tower with no disks.")
    continue # Ask player again for their move.
elif len(towers[toTower]) == 0:
    # Any disk can be moved onto an empty "to" tower:
    return fromTower, toTower
elif towers[toTower][-1] < towers[fromTower][-1]:
    print("Can't put larger disks on top of smaller ones.")
    continue # Ask player again for their move.
```
